

Missão: Sua Própria Pokédex, Sem Pokémons

Replique a arquitetura construída em aula — MVVM, Retrofit e Coroutines — consumindo uma API pública diferente.

Contexto

Em aula, construímos juntos uma Pokédex que consulta a PokéAPI, organizada em camadas (**Model / ViewModel / View**), com estados de tela representados por uma `sealed interface` e chamadas de rede assíncronas via Coroutines. Nesta atividade, você vai provar que entendeu a arquitetura — não apenas copiou o código — construindo o mesmo ecossistema para dados diferentes.

Especificação da Missão

Escolha **uma** API REST pública e gratuita e construa um novo aplicativo Android do zero:

ViaCEP API

Rick and Morty API

The Dog API

Requisitos Obrigatórios

- Arquitetura em camadas:** pacotes `model`, `network`, `viewmodel` e `ui` — seguindo exatamente a mesma separação vista em aula.
- Camada de rede com Retrofit:** uma `interface` anotada com `@GET` e um cliente Retrofit configurado com a URL base da API escolhida.
- Mapeamento do JSON:** `data class` representando a resposta da API, usando `@SerializedName` onde o nome do campo no JSON for diferente do nome em Kotlin.
- Gerenciamento de estado:** uma `sealed interface` (ou `sealed class`) cobrindo os estados possíveis da tela — no mínimo carregando, sucesso e falha.
- ViewModel isolando a UI da rede:** a tela nunca chama o Retrofit diretamente; ela apenas lê o estado exposto pelo ViewModel e dispara ações por callback.
- Assincronismo correto:** chamadas de rede em funções `suspend`, disparadas dentro de `viewModelScope`, nunca na Main Thread.
- Interface em Jetpack Compose:** exibindo os dados retornados e permitindo alguma interação do usuário (busca, navegação entre itens, ou equivalente).
- Tratamento de erro visível:** se a busca falhar ou o item não existir, o app mostra uma mensagem — nunca fecha nem trava.
- Permissão de internet** declarada no `AndroidManifest.xml`.
- Repositório no GitHub** com um `README.md` explicando o que o app faz e como rodar.

Rubrica de Correção

Critério	Peso	O que será verificado
Arquitetura em camadas (MVVM)	25%	Separação real entre model, network, viewmodel e ui; a UI não conhece o Retrofit.
Camada de rede funcional	20%	Retrofit configurado, JSON mapeado corretamente, dados chegando na tela.
Gerenciamento de estado	20%	<code>sealed interface / class</code> tratando os estados de forma explícita, sem combinações inválidas.
Assincronismo e tratamento de erro	20%	Coroutines aplicadas corretamente; falha de rede não trava nem fecha o app.
Versionamento e README	15%	Repositório público no GitHub com documentação mínima.

Checklist de Autoavaliação

Antes de entregar, confirme:

- ☐ Minha tela não importa nada do pacote `network` — só conversa com o ViewModel.
- ☐ Testei uma busca inválida de propósito e o app mostrou uma mensagem, sem fechar.
- ☐ Desliguei a internet, tentei buscar, e o app tratou o erro corretamente.
- ☐ Girei o emulador e os dados carregados continuaram na tela.
- ☐ Rodei o app do zero (cold start) e ele funcionou sem crashar.
- ☐ Meu `README.md` explica em 3-4 frases o que o app faz.

Instruções de Entrega

Suba o projeto completo em um repositório público no GitHub e entregue o link do repositório na plataforma indicada pelo professor. O `README.md` deve conter: nome da API escolhida, principais telas/funcionalidades, e como rodar o projeto no Android Studio.